

WuWei ホーム画面仕様

Table of Contents

1. 目的	1
2. Resource library の読み込み	1
3. ホームカードに変換される項目	2
4. 検索と日付	2
5. ノートへの挿入	2
6. Office 文書と PDF preview	3
7. Contents axis との関係	3
8. エラー時の確認項目	4

1. 目的

ホーム画面は、保存済みノートと Resource library を一覧し、ノート編集画面へ読解対象となるコンテンツを挿入するための入口である。

WuWei のノート本体は、`groups[]`、`nodes[]`、`links[]` で構成される読解ネットワークである。ホーム画面の Resource library は、そのネットワークに `Content` ノードとして取り込む文書、映像、Web ページ、画像などを管理するための入口である。

現在の実装では、リソース一覧の正本は `upload/**/manifest.json` ではなく、次の `resource.json` である。

```
data/{user_uuid}/resource/YYYY/MM/DD/{resource_uuid}/resource.json
```

`resource.json` は、実体ファイルを `storage.files[]` と `storage.manifest` で `upload/` 側に参照する。

2. Resource library の読み込み

`list-resource.cgi` と `search-resource.cgi` は、ユーザー領域の `resource/**/resource.json` を走査する。

```
data/{user_uuid}/resource/YYYY/MM/DD/{resource_uuid}/resource.json
```

表示対象外になる代表例は次である。

- `type` が `Resource` ではない。
- `home.hidden` または `hiddenFromHome` が `true`。
- `storage.files[]` に表示・ダウンロードに必要なパスがない。

- `storage.manifest.path` が壊れている。

3. ホームカードに変換される項目

`resource.json` はホーム画面用に次のようなカード情報へ変換される。

カード項目	主な変換元
<code>id</code>	<code>resource.id</code>
<code>label, name, description</code>	<code>title, label, description, origin.originalName</code>
<code>contenttype</code>	<code>mimeType, media.mimeType</code>
<code>thumbnail_url</code>	<code>storage.files[]</code> の <code>role=thumbnail</code>
<code>preview_url</code>	<code>storage.files[]</code> の <code>role=preview</code> または PDF の <code>role=original</code>
<code>download_url</code>	<code>storage.files[]</code> の <code>role=original</code>
<code>value.resource</code>	Content ノードに埋め込む Resource 定義
<code>value.viewerType</code>	PDF、image、video、Office preview などのビューア種別

ファイル URL は原則として `load-file.cgi` 形式で生成する。

```
server/load-  
file.cgi?area=upload&path=YYYY/MM/DD/{upload_uuid}/thumbnail.jpg&user_id={user_uuid}
```

4. 検索と日付

検索対象は `resource.json` である。登録日や更新日は、次の順に利用できる情報から判断する。

- `audit.createdAt`
- `audit.lastModifiedAt`
- `storage.files[].updatedAt`
- `resource.json` のファイル更新時刻
- パス中の `YYYY/MM/DD`

5. ノートへの挿入

ホーム画面からコンテンツを選択すると、ノート上に `Content` ノードを作成する。

現在の通常保存では、Content ノードは top-level `resources[]` への参照ではなく、ノード内に compact resource を保持する。

```
{  
  "type": "Content",  
  "shape": "THUMBNAIL",  
}
```

```

"label": "document.pdf",
"resource": {
  "kind": "upload",
  "id": "_resource_uuid",
  "mimeType": "application/pdf",
  "storage": {
    "manifest": {
      "area": "upload",
      "path": "YYYY/MM/DD/_resource_uuid/manifest.json"
    },
    "files": []
  }
},
"resourceView": {
  "mode": "default"
}
}

```

読み込み互換のため、top-level `resources[]` と `resourceRef` の形式も扱えるが、現在の保存形式では `resources[]` は空になりやすい。

6. Office 文書と PDF preview

Office 文書は、アップロード時に `preview.pdf` を生成できる場合がある。

この場合、表示と Contents axis は `storage.files[]` の `role=preview` を使う。

```

{
  "contents": {
    "type": "pdf",
    "axis": {
      "unit": "page",
      "nodeType": "page"
    },
    "pageCount": 27,
    "sourceRole": "preview"
  }
}

```

PDF 原本の場合は、`sourceRole` は `original` になる。

7. Contents axis との関係

PDF または Office preview PDF の Resource が `pageCount` を持つ場合、ノート上で Contents axis を作成できる。

Contents axis のページ参照ノードは `PageMarker` である。

```
{
  "type": "PageMarker",
  "topicKind": "contents-page",
  "contentsRef": "_contents_group_uuid",
  "documentRef": "_content_node_uuid",
  "pageNumber": 7,
  "label": "p.7"
}
```

PageMarker から文書を開く場合は、対象 PDF URL に `#page=7` のようなページ指定を付ける。

8. エラー時の確認項目

ホームに表示されない場合は、次を確認する。

- `resource/YYYY/MM/DD/{resource_uuid}/resource.json` が存在する。
- `resource.json` が有効な JSON である。
- `type` が `Resource` である。
- `storage.files[]` に `role=original` がある。
- サムネイル表示には `role=thumbnail` がある。
- Office preview 表示には `role=preview` がある。
- `load-file.cgi` が該当 `area` と `path` を返せる。